

PWANINET Architecture Map

This document gives a visual map of the system architecture and core data relationships.

1) System Context

```
```mermaid
flowchart LR
 U[User Browser] -->|HTTP/HTMX| DJ[Django App]
 DJ -->|ORM| DB[(SQLite Database)]
 DJ -->|Serve Static| ST[Static Files]
 DJ -->|Serve Media| MD[Media Files]
...

```

### ## 2) Request Lifecycle

```
```mermaid
flowchart TD
    A[Incoming Request] --> B[pwaninet/urls.py]
    B --> C[core/urls.py or Django auth URLs]
    C --> D[Middleware Stack<br/>Sessions/Auth/CSRF/Messages/WhiteNoise]
    D --> E[core/views.py function-based view]
    E --> F[ORM access via core/models.py]
    F --> G[(SQLite)]
    E --> H[Template render]
    H --> I[HTML or Partial HTML Response]
...

```

3) Layered Architecture

```
```mermaid
flowchart TB
 subgraph Presentation
 T1[core/templates/*.html]
 T2[core/templates/partials/*.html]
 T3[HTMX + Bootstrap]
 end

 subgraph Application
 V1[core/views.py]
 F1[core/forms.py]
 U1[core/urls.py]
 CP[core/context_processors.py]
 end

 subgraph Domain
 M1[core/models.py]
 S1[core/signals.py]
 end

 subgraph Infrastructure
 P1[pwaninet/settings.py]
 P2[pwaninet/urls.py]
 P3[manage.py]
 P4[core/admin.py]
 P5[management commands]
 end

 T1 --> V1
 T2 --> V1
 T3 --> V1
 V1 --> M1
 F1 --> V1

```

```

U1 --> V1
CP --> T1
S1 --> M1
P1 --> U1
P2 --> U1
P3 --> P1
P4 --> M1
...

```

#### ## 4) Core Domain Model Relationships

```

```mermaid
erDiagram
    USER ||--o{ POST : creates
    USER ||--o{ FOLLOW : follower
    USER ||--o{ FOLLOW : following
    USER ||--o{ LIKE : likes
    USER ||--o{ NOTIFICATIONS : receives
    USER ||--o{ GROUPS : creates
    USER }o--o{ GROUPS : member_of

    COURSE ||--o{ UNIT : has
    YEAR ||--o{ UNIT : has
    UNIT ||--o{ POST : tagged_in
    COURSE ||--o{ POST : tagged_in
    GROUPS ||--o{ POST : posted_in
    POST ||--o{ LIKE : liked_by
...

```

5) Feature-to-File Map

- Feed and posting: `core/views.py`, `core/templates/home.html`,
`core/templates/partials/post\_card.html`
- Search: `core/views.py`, `core/templates/search\_results.html`,
`core/templates/partials/search\_results\_content.html`
- Groups and membership: `core/views.py`, `core/templates/groups\_\*.html`,
`core/models.py`
- Notifications: `core/views.py`, `core/context\_processors.py`,
`core/templates/partials/notification\_badge.html`
- Authentication: `pwaninet/urls.py`, `core/forms.py`, `core/views.py`
- Data seeding: `core/management/commands/populate\_units.py`,
`core/management/commands/populate\_posts.py`

6) Runtime Entrypoints

- Local/dev CLI: `manage.py`
- WSGI deployment: `pwaninet/wsgi.py`
- ASGI deployment: `pwaninet/asgi.py`

7) Notes

- The app is a server-rendered Django monolith with interactive partial updates.
- No dedicated REST API or async task queue is currently evident from project structure.